# 体に付けるセンサーの作り方

XIAO nRF52840 ＋ 6軸センサー（LSM6DSV16X）で、動きの値を Bluetooth で Mac に送るまで。1個目はブレッドボードで、2個目からは空中配線・電池つきで体に付ける

2026年10月14日版　／　ファイル一式：`~/Developer/run-form-coach/hardware/sensor/`

## 0. この手順書でやること

6か所（両手首・両足首・首・胸）に付けるセンサーを作ります。手順1〜8 は**最初の1個**で、ブレッドボードの上で、Mac の USB かモバイルバッテリーで動かします。**2個目から**は「2個目からの作り方」の章で、ブレッドボードを使わずに線を直接はんだ付けし、電池をつけて体に付けます。

| 手順1 | 手順2 | 手順3 | 手順4 | 手順5 | 手順6 | 手順7 |
|---|---|---|---|---|---|---|
| Mac の準備 | XIAO だけで書き込み確認 | はんだ付け | 配線 | センサーの値を見る | Bluetooth で Chrome へ | モバイルバッテリーで |

> **失敗しないための考え方**：1つの手順が通ってから次へ進みます。特に**手順2（はんだ付けの前に、XIAO 単体で書き込みを確かめる）**を飛ばさないでください。XIAO が初めから不良でも、はんだ付けする前なら交換を頼めます。

### この手順書で確かめ済みのこと・まだのこと

| 確かめ済み（実機で） | まだ（2個目を作るときに確かめる） |
|---|---|
| • 1個目（ブレッドボード）で手順1〜7 が通った<br>• Bluetooth で Mac の Chrome に 1秒に約100サンプル、抜け 0 件で届いた<br>• センサー基板の回路図（秋月電子）から、ジャンパーとピンの役割を確認 | • 加速度の範囲を ±16 g にし、通知に範囲と電池の電圧を足した今のプログラム（コンパイルとパーサのテストまでは通っている）<br>• 電池の電圧の読み取り、つながらないまま10分で眠る動き<br>• 空中配線・電池つきの作り方そのもの<br>• 110mAh の電池で何時間動くか |

# 1. そろえるもの

## 部品（秋月電子で1回の注文）

| 部品 | 数 | 値段（税込） | 販売コード | メモ |
|---|---|---|---|---|
| Seeed XIAO nRF52840（**無印**） | 2 | 3,560円 | 117341 | 技適 211-220207。「Plus」「Sense」ではない無印を選ぶ |
| 6軸IMUセンサーモジュール AE-LSM6DSV16X | 1 | 990円 | 130950 | ピンヘッダー付属 |
| ピンヘッダー 1×40（PH-1x40SG） | 2 | 70円 | 100167 | XIAO 用。7ピンずつ切って使う |
| ブレッドボード＋ジャンパーワイヤーセット（EIC-102J） | 1 | 1,280円 | 102314 | |
| **合計** | | **約5,900円**＋送料 | | |

## ほかに用意するもの

- **USB-C のケーブル**（XIAO と Mac をつなぐ）：**「充電・転送」「データ転送対応」**と書いてあるもの。**「充電専用」は使えません**（Mac から XIAO が見えず、書き込めない）。ダイソーの 330円「USB Type-C 充電・転送ケーブル」でも足ります。Mac の差し込み口が楕円形なら両端 USB-C を選びます。
- はんだごて、はんだ、テスター、ニッパー（手元にあるもの）
- モバイルバッテリー（手順7で使う）
- Mac（Apple シリコン）、Homebrew、Google Chrome（この Mac にはすべて入っています）

# 2. 安全の注意（先に読む）

> **センサーに 5V をつながない**：センサーは 3.3V 専用です。XIAO の **5V** ピンにつなぐと壊れます。必ず **3V3** ピンにつなぎます。

> **センサー基板のジャンパーは「6A」と「I2C」だけ**：「6B」と「6A」を両方ふさぐと、アドレスの線がぶつかります。「CS」をふさぐと I2C で動かなくなります。裏の「SDX」「SCX」の四角もふさぎません。

> **電源を入れる前に、テスターでショートを確かめる**：はんだ付けの後、3V3 と GND の間がつながっていない（導通しない）ことを確かめます。

- XIAO の USB-C の差し込み口は小さくて外れやすいので、ケーブルを横にこじらない。
- 基板を触る前に金属に触れて、静電気を逃がす。

## 手順1. Mac の準備（約15分、最初に1回だけ）

ターミナルで次を実行します。

```
cd ~/Developer/run-form-coach/hardware/sensor
tools/setup.sh
```

このスクリプトは、次の4つをします。

1. **Rosetta** を入れる：XIAO 用のコンパイラーが Intel の Mac 向けにしか作られていないため、Apple シリコンの Mac では必要です（この Mac には、2026-10-09 に入れました）。
2. **arduino-cli**（Arduino のコマンド版）を入れる（入れ済み）。
3. **XIAO nRF52840 のボードのパッケージ**（Seeed nRF52 Boards）を入れる（入れ済み）。
4. ボードのパッケージが `python` という名前を呼ぶのに、Mac には `python3` しかないので、`tools/bin/python` でつなぐ。

> **なぜ Arduino IDE（画面で操作するほう）を使わないか**：上の 1 と 4 の問題で、Mac の Arduino IDE ではコンパイルが止まる可能性が高いからです。この手順書のコマンドは、作者の Mac で実際に通してあります。

> **成功のしるし**：最後に「== 準備ができました」と出る。

## 手順2. XIAO だけで書き込みを確かめる（はんだ付けの前に）

1. XIAO を、データ転送対応の USB-C ケーブルで Mac につなぐ。
2. ターミナルで書き込む：

   ```
   tools/flash.sh 01_blink
   ```

3. XIAO の文字を見る（終わるときは Ctrl+C）：

   ```
   tools/monitor.sh
   ```

> **成功のしるし**：XIAO の小さな LED が1秒ごとに点滅し、モニタに `XIAO is alive: 0, 1, 2…` と出る。

**「XIAO が見つかりません」と出たら**：①ケーブルが充電専用でないか、②XIAO のリセットボタン（USB の横の小さなボタン）を**素早く2回**押して書き込み待ちにしてから、もう一度 `tools/flash.sh 01_blink`。

**2個目の XIAO でも同じことをします。**どちらも動けば、XIAO は良品です。

## 手順3. はんだ付け

### 3-1. XIAO にピンヘッダー

- XIAO は、**両側のふちに7個ずつ、14個の穴すべて**にピンを付けます。ピンヘッダー（1×40）をニッパーで**7ピンずつ2本**に切る。
- XIAO を**部品の面を上**（USB-C の差し込み口が見える側）にして、ピンヘッダーを**先に XIAO の穴へ下から通し**ます（黒い樹脂は XIAO の下側、短いほうのピンが上に少し出る）。
- そのまま、ブレッドボードの**まん中の溝をまたぐ向き**で挿す（左右の列が溝の両側に分かれる）。こうするとピンがまっすぐに保たれ、手を離してもはんだ付けできます。
- 上から14か所をはんだ付けする。はじめに対角の2か所を付けて、XIAO が浮いていない・傾いていないのを見てから残りを付けます。

### 3-2. センサー基板にピンヘッダー

- 付属のピンヘッダーは、**最初から3ピンずつ4本に分かれている**ことがあります。そのときは切らずにそのまま使います。1本につながっているときは、ニッパーで**3ピンずつ**に切り離す。
- 基板の4か所の3ピンの穴（左右に2組ずつ）すべてに付けると、ブレッドボードの上で安定します。XIAO と同じく、先に基板に通してからブレッドボードに挿してはんだ付けします。

### 3-3. センサー基板のジャンパーをふさぐ（大事）

この基板は、**ジャンパーをふさがないと I2C で動きません。**小さな四角い接点のすき間を、はんだで橋渡しします。基板に「J1」などの番号は印字されていません。**下のふちに、左から「6B」「6A」「I2C」「CS」**と印字された接点の組があります。

| 基板の印字 | 役割（回路図の番号） | どうする |
|---|---|---|
| **6A** | I2C のアドレスを **0x6A** にする（J3）。接点2つ | **ふさぐ**（すき間1つ） |
| **I2C** | SDA と SCL のプルアップ抵抗（10kΩ）をつなぐ（J1）。**接点が3つ**並んでいる | **ふさぐ（すき間2つとも）** |
| 6B | アドレスを 0x6B にする（J2） | ふさがない（6A と同時は不可） |
| CS | CS を GND につなぐ（J4、SPI 用） | **ふさがない**（ふさぐと I2C で動かない） |
| 裏の「SDX」「SCX」の四角 | 補助の端子を GND につなぐ（J5・J6） | ふさがない |

CS は基板の上で 10kΩ で 3.3V に引き上げられているので、何もつながなければ I2C になります。プログラムは 0x6A と 0x6B の両方を探すので、6A の代わりに 6B をふさいでも動きます。

### 3-4. 確かめる

テスターで：

☐ XIAO の 3V3 と GND の間が導通しない

☐ センサーの 3V3 と GND の間が導通しない

☐ 「6A」と「I2C」（すき間2つとも）がふさがっている（両側が導通する）。「6B」「CS」はふさがっていない

☐ 隣どうしのピンが、はんだでくっついていない

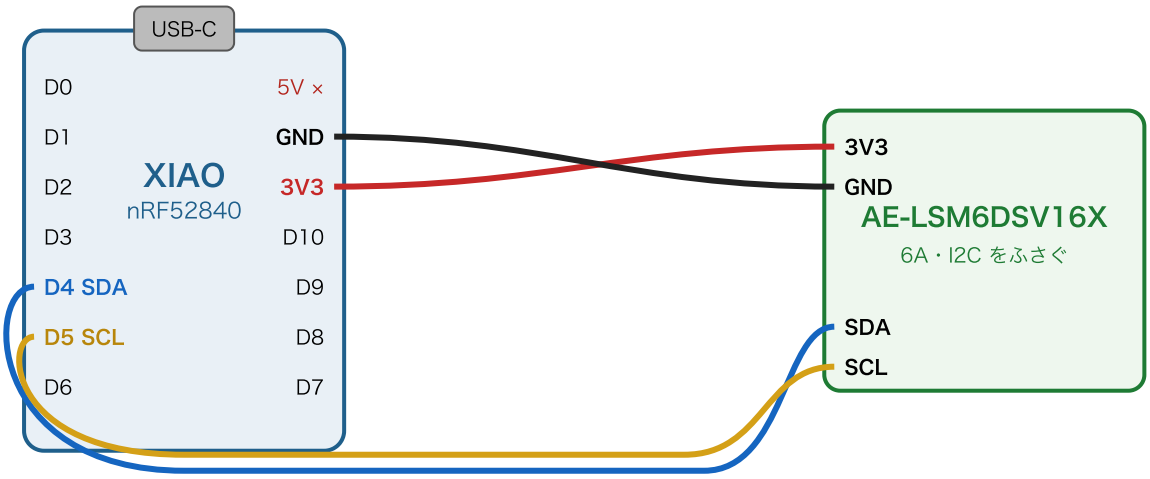
**テスターが無くても進めるとき**（目で見る）：明るいところでスマホのカメラで拡大して、①「6A」のすき間と「I2C」のすき間2つが、つやのあるはんだの山でつながっている、②「6B」「CS」と裏の「SDX」「SCX」には、すき間が見えている、③XIAO とセンサーのピンの根元で、隣のピンとはんだがつながっていない、を確かめる。ショートは目では見落としやすいので、手順5で USB をつないだら XIAO とセンサーを指で触り、**熱くなったらすぐ抜く**。

## 手順4. 配線（ブレッドボード）

つなぐのは**4本だけ**です。センサー基板のピンの名前は、**基板に印刷された文字を見て**確かめてください。

| XIAO のピン | → | センサー基板のピン | 線の色（おすすめ） |
|---|---|---|---|
| **3V3**（右側の上から3番目） | → | **3V3** | 赤 |
| **GND**（右側の上から2番目） | → | **GND** | 黒 |
| **D4 = SDA**（左側の上から5番目） | → | **SDA** | 青 |
| **D5 = SCL**（左側の上から6番目） | → | **SCL** | 黄 |

XIAO のピンの位置は、部品の面を上・USB-C を上にして見たときのものです。左側は上から D0〜D6、右側は上から 5V・GND・3V3・D10・D9・D8・D7 の順です。**5V（右側のいちばん上）には何もつなぎません。**センサーの CS・SDO/SA0・INT1・INT2・SDX・SCX・SDO_AUX・OCS_AUX はつなぎません。




図：つなぐのは 3V3・GND・SDA・SCL の4本。センサー側のピンの実際の並びは、基板の印刷を見て合わせる。

> **つなぐ前にもう一度**：赤い線が XIAO の **3V3**（5V ではない）につながっているか、目で確かめる。

## 手順5. センサーの値を見る

```
tools/flash.sh 02_check_lsm6dsv
tools/monitor.sh
```

**成功のしるし**：次のように出る。

```
Start!
LSM6DSV16X found. address: 0x6A
WHO_AM_I = 0x70
LSM6DSV16X initialized.
ax[g], ay[g], az[g], gx[dps], gy[dps], gz[dps]
0.01, -0.02, 1.00, 0.35, -0.21, 0.07
```

机に置いたまま動かさないと、加速度のどれか1つが約 **1.00**（重力）、ほかの2つは約 0、ジャイロ（後ろの3つ）はほぼ 0。基板を傾けると加速度が、回すとジャイロが大きく変わる。

| 出たもの | 意味と、やること |
|---|---|
| `LSM6DSV16X not found` が繰り返される | センサーが見えていない。①4本の配線、②「6A」と「I2C」（すき間2つとも）がふさがっているか、③SDA と SCL が逆になっていないか、④センサーの 3V3 に電圧が来ているか（テスターで約3.3V）を確かめる |
| `WHO_AM_I = 0x...` が 70 以外 | 別のチップか、通信が乱れている。配線を短くし、はんだ付けを見直す |
| 値がずっと 0 のまま | 初期設定が効いていない。XIAO のリセットボタンを1回押して、もう一度見る |

## 手順6. Bluetooth で Mac の Chrome に送る

1. Bluetooth で送るプログラムを、**番号 1** を付けて書き込む（名前が RFC-IMU-1 になる）：

   ```
   tools/flash.sh 03_ble_imu 1
   ```

2. 受信画面を開く（Chrome が自動で開きます。終わるときはターミナルで Ctrl+C）：

   ```
   tools/viewer.sh
   ```

3. 画面の**「接続」**を押し、一覧の **RFC-IMU-1** を選んで「ペア設定」。

**成功のしるし**：加速度とジャイロの数字が動き、**「受信：約100 サンプル/秒」**で、**「抜け：0 件」**のまま増えない。**「加速度の範囲：±16 g」**と出る。

1回の通知に何サンプルかまとめて届くので、横に出る「通知 ○ 回/秒」は 100 より少なくて構いません。プログラムの `SAMPLE_HZ` を 50 にして書き込んだときは、約50 サンプル/秒が正常です。

- **一覧に何も出ないとき**：Mac の「システム設定 → プライバシーとセキュリティ → Bluetooth」で、Google Chrome を許可する。Mac の Bluetooth が入っているかも確かめる。
- **XIAO の LED が速く点滅し続けるとき**：センサーが見つかっていない。手順5に戻る。

- **10分つながないと眠ります**（電池の持ちのため）。一覧に出なくなったら、XIAO のリセットボタンを**1回**押すか、USB をつなぎ直すと起きます。
- 受信画面は、Web Bluetooth の決まりで `https` か `localhost` でしか動きません。`viewer.html` をダブルクリックで開くのではなく、必ず `tools/viewer.sh` から開きます。

## 手順7. モバイルバッテリーで動かす

1. XIAO を Mac から外し、モバイルバッテリーにつなぐ（書き込んだプログラムはそのまま残っています）。
2. Mac の受信画面で、もう一度「接続」。
3. **5分そのまま**置いて、切れないか見る。

> **モバイルバッテリーが勝手に切れることがある**：XIAO はとても少ない電気で動くので、バッテリーが「充電が終わった」と思って 30秒〜1分で切れることがあります。「低電流モード」（小型機器モード）があれば使ってください。

## 手順8（できれば）. トレッドミルで記録する

- XIAO とブレッドボードとモバイルバッテリーを、ウエストポーチなどに入れて体に固定する（ぶら下げると揺れが雑音になる）。
- 受信画面の「記録を始める」→ 走る →「記録を止める」→「CSVで保存」。
- CSV には、時刻（ミリ秒）、加速度 3軸（g）、ジャイロ 3軸（dps）が入ります。1秒に約100行（1サンプル1行）です。ファイル名にセンサーの名前（RFC-IMU-2 など）が入ります。

## うまくいかないとき（まとめ）

| 起きたこと | 確かめること |
|---|---|
| コンパイルで `bad CPU type in executable` | Rosetta が入っていない。`tools/setup.sh` をもう一度 |
| コンパイルで `exec: "python": executable file not found` | `tools/bin/python` がない。`tools/setup.sh` をもう一度。必ず `tools/flash.sh` から書き込む |
| 「XIAO が見つかりません」 | 充電専用ケーブルでないか。リセットボタンを素早く2回押してから再実行 |
| 書き込んだら動かなくなった、Mac から見えない | リセットボタンを素早く2回押すと書き込み待ちに戻る。それでも駄目なら USB を抜き差し |
| センサーが見つからない | 手順5の表を見る |
| Chrome の一覧に出ない | Chrome の Bluetooth の許可。XIAO の LED が速く点滅していないか |
| 受信が 1秒に100サンプルより大きく少ない、「抜け」が増えていく | Mac と XIAO を近づける。ほかの Bluetooth 機器を減らす。それでも直らなければ `SAMPLE_HZ` を 50 にして書き込み直す |
| 受信画面に「形の違うデータが届いています」 | XIAO のプログラムが古い。`tools/flash.sh 03_ble_imu 番号` で書き込み直す |
| さっきまで一覧に出ていたのに出なくなった | 10分つながらなかったので眠った。リセットボタンを1回押すか、USB をつなぐ |
| 電池つきで、USB を外すと動かない | 電池が空か、BAT の線が外れている。USB で充電して（赤い LED が消えるまで）から、もう一度 |

## 2個目からの作り方（空中配線・電池つき）

2個目からは体（両手首・両足首・首・胸）に付けるので、ブレッドボードもピンヘッダーも使わず、**線を XIAO に直接はんだ付け**し、電池をつけて、小さく重ねて固定します。1個目で通った手順（1・2・5・6）はそのまま使います。

| A | B | C | D | E | F |
|---|---|---|---|---|---|
| XIAO だけで書き込み確認 | センサーのジャンパー | コードをはんだ付け | 名前を付けて書き込む | 電池をつなぐ | 重ねて体に付ける |

> **順番の理由**：電池は**いちばん最後**につなぎます。電池をつなぐと、はんだ付けの途中でも XIAO に電気が入るからです。また、電池をつなぐ前に今の `03_ble_imu` を書き込んでおきます（下の「P0.14」の注意）。

### 1個分にそろえるもの

| もの | メモ |
|---|---|
| Seeed XIAO nRF52840（無印） | 1個目と同じ |
| AE-LSM6DSV16X ＋ 付属のコード | 白い小さなコネクター（Qwiic、SH 1mm 4ピン）から、黒・赤・青・黄の4本に分かれて、先がオスピンになっているコード |
| リポ電池 OCTelect 401030 110mAh（3.7V） | 小さい白いコネクター付き。保護回路付き。1個に1本 |
| 熱収縮チューブ φ1.5 | 4本 × 1cm ほど |
| 厚手の両面テープ（クッションタイプ） | XIAO とセンサーと電池を重ねて貼る。厚みで、はんだの出っぱりが当たらないようにする |
| ビニールテープかカプトンテープ | 電池の線の付け根を押さえる・むき出しの線を一時的に包む |
| ミニチャック袋（ポリ袋）、面ファスナーのバンド | 汗よけと、体への固定。**金属やアルミの袋は使わない**（Bluetooth の電波が通らない） |
| マスキングテープとペン | 番号と場所のラベル |

### A. XIAO だけで書き込みを確かめる

手順2と同じです。はんだ付けの前に `tools/flash.sh 01_blink` で LED が点滅するのを見ておきます。

### B. センサーのジャンパー（1個目と同じ）

下のふちの**「6A」**と**「I2C」**をはんだでふさぎます。**I2C は接点が3つで、すき間2つとも**ふさぎます。**「6B」「CS」、裏の「SDX」「SCX」はふさぎません。**くわしくは 3-3 の表。

## C. コードを XIAO にはんだ付けする

**センサー側は、白いコネクターを挿すだけ**です。XIAO 側を、ピンヘッダーなしで穴に直接はんだ付けします。

| コードの色 | → | XIAO の穴 | XIAO の上での位置（部品の面を上、USB-C を上にして） |
|---|---|---|---|
| **黒** | → | **GND** | 右側の上から2番目 |
| **赤** | → | **3V3（5V ではない）** | 右側の上から3番目 |
| **青** | → | **D4**（SDA） | 左側の上から5番目 |
| **黄** | → | **D5**（SCL） | 左側の上から6番目 |

色は Qwiic の決まり（黒 GND・赤 3.3V・青 SDA・黄 SCL）です。秋月の商品ページは、この基板のコネクターを「Qwiic 準拠」としています。

1. コードを、**コネクターから 5cm ほど**のところで切る（4本まとめて切ってよい。電池の線とちがって電気は来ていない）。体に付けないで試すだけなら、切らずにオスピンのまま XIAO の穴に挿してはんだ付けしてもよいが、長さが余ってかさばる。
2. 4本を少し裂いて離し、それぞれ**先の皮を 3mm むく**。細い線なので、より合わせてから先にはんだを少しのせる（予備はんだ）。
3. **熱収縮チューブ（φ1.5、1cm）を、はんだ付けの前に線に通しておく。**付けた後では通せない。
4. 線を XIAO の**部品の面から**穴に差し、部品の面ではんだ付けする。裏に突き出た先は、ニッパーで短く切る（背中合わせに重ねたときに当たらないように）。
5. チューブを根元までずらし、はんだごての横腹を近づけて縮める（こて先を直接当てない）。
6. 手順3-4 と同じく、目かテスターで、隣どうしがくっついていない・3V3 と GND がつながっていないのを確かめる。
7. USB をつないで `tools/flash.sh 02_check_lsm6dsv` と `tools/monitor.sh`（手順5）で、センサーが見えるのを確かめる。

## D. 名前を付けて書き込む

何個目かの番号を付けて書き込むと、その番号の名前（RFC-IMU-2 など）になります。プログラムを書き換える必要はありません。

```
tools/flash.sh 03_ble_imu 2     # 2個目。3個目なら 3、…
```

`tools/viewer.sh` で、一覧に **RFC-IMU-2** が出て、1秒に約100サンプル・抜け 0 件・「加速度の範囲：±16 g」になるのを確かめます（手順6と同じ）。

**番号と付ける場所**は頭から順に決めています。ラベルを貼り、袋の外にも同じラベルを貼ると、付けるときに迷いません。番号は書き込むときに決まるので、付ける場所を替えるときは、その番号で書き込み直します。

| 名前 | 付ける場所 |
|---|---|
| RFC-IMU-1 | 頭（帽子の後ろ、または首の付け根） |
| RFC-IMU-2 | 胸 |
| RFC-IMU-3 | 左手首 |
| RFC-IMU-4 | 右手首 |
| RFC-IMU-5 | 左足首 |
| RFC-IMU-6 | 右足首 |

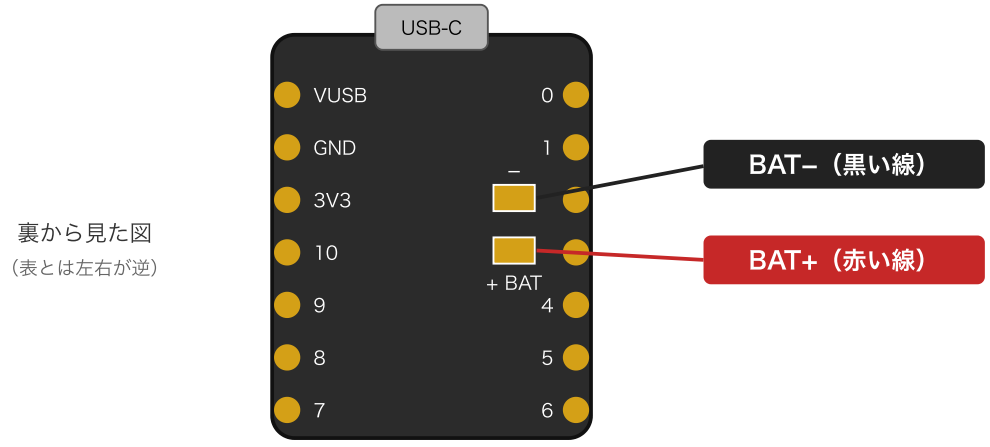
ラベルには、体に付けたときの**上（頭の方向）を指す矢印**も書いておきます。毎回同じ向きに付けると、記録どうしを比べられます。

## E. 電池をつなぐ

> **プラスとマイナスを逆につなぐと、その瞬間に XIAO が壊れます**：**赤 ＝ BAT+、黒 ＝ BAT−**。電池の保護回路は、逆につないだのを防いではくれません。**3V3 や 5V のピンにはつなぎません**（必ず裏の BAT の接点）。

> **電池のコネクターは、線を1本ずつ切る**：2本いっしょにニッパーで挟むと、刃の上でプラスとマイナスがつながってショートします（火花・発熱）。1本切ったら、その先をすぐテープで包んでから、もう1本を切ります。

**BAT の接点の位置**（Seeed の公式資料のピン配置図「XIAO nRF52840 Back」で確認）：XIAO を**裏返して**、USB-C を上にして見ると、**右側のふち（表の D0〜D6 の裏側）の、上から3つ目と4つ目のピンの内側**に、小さな四角い接点が縦に2つ並んでいます。**上が BAT−、下が BAT+**。基板に「−」「+ BAT」と印刷されています。




図：XIAO nRF52840（無印）の裏。Seeed の「XIAO_nRF52840_back_pinout」をもとに描いたもの。Plus・Sense Plus は位置が違う。

1. **電池の＋−を確かめる**：テスターがあれば、電池の赤と黒の線の電圧を測り、赤をテスターの赤に当てて **+3.x V**（マイナスでない）になるのを見る。テスターが無いときは、**線の色**と、**電池本体の、線が**

**出ている所の近くの「＋」「−」（「B+」「P+」なども同じ意味）の印刷**を見て、赤が＋につながっているのを確かめる。どちらかがはっきりしないときは、つながない。

2. XIAO の USB を外す。**電池がふくらんでいたら使わない。**
3. コネクターの根元で、**線を1本ずつ**切る（上の注意）。線は 3〜4cm 残す。
4. XIAO の裏の BAT の2つの接点に、先にはんだを少しのせておく。
5. **黒（BAT−）から**付ける：黒の先を 2mm むいて予備はんだし、BAT− の接点にのせて、こてで温めて付ける。この間、赤の先はテープで包んだまま。
6. **赤（BAT+）**：テープを外し、同じように 2mm むいて BAT+ に付ける。付けた瞬間から XIAO が電池で動き出す。
7. 2本の線を、XIAO の裏にテープで押さえる（線を引っぱると接点ごとはがれるので、根元に力がかからないように）。
8. Mac の受信画面でつながるのを確かめる（電池だけで動いている）。**「電池：3.xx V」**と出る。

## 充電

- **XIAO を USB-C で Mac か USB 充電器につなぐだけ**で充電します。充電の間は、XIAO の USB の横の**赤い小さな LED**（充電の LED）が点き、満タンになると消えます。
- 充電の電流は、何もしなければ**約 50mA** です（Seeed の資料：P0.13 を何もつながない入力にすると約 50mA、出力の LOW にすると約 100mA。ボードのパッケージは起動時に P0.13 を入力にし、このフォルダーのプログラムは P0.13 を触らない）。110mAh の電池には約 0.45C で、無理のない速さです。空から満タンまで2時間半〜3時間ほど。**100mA には上げません**（110mAh には約 0.9C で、余裕がない）。
- **P0.14 の注意（Seeed の公式資料）**：P0.14 は電池の電圧を読む道を開け閉めするピンで、**HIGH にしたまま充電すると、電圧を読むピン（P0.31）が上限の 3.6V に届いて壊れるおそれがある**ので、いつも LOW にしておくように、とあります。ボードのパッケージは起動時に P0.14 を HIGH にするので、このフォルダーのプログラム（01・02・03）は、**起動してすぐ P0.14 を LOW にします。**ほかの人のプログラムを、電池をつないだ XIAO に書き込まないでください。
- **充電中は目を離さない。**燃えるものの近くで充電しない。袋に入れたまま・体に付けたまま充電しない。
- **ふくらんだ電池、熱くなる電池は使わない。**落とした・つぶした電池も使わない。
- 電池の電圧は受信画面に出ます：**4.2V で満タン、3.5V を切ったら充電。**USB につないでいる間は、充電の電圧（4V 台）が出ます。

## 眠る・起きる（スイッチの代わり）

電池をはんだ付けすると、スイッチが無いので、ずっと電気が流れます。そこで `03_ble_imu` は、**だれともつながらないまま10分たつと眠ります**（センサーも止める）。起こすには、**XIAO のリセットボタンを1回押す**か、**USB をつなぎます。**走る前に起こしてから袋に入れると楽です。眠っている間もわずかに電気を使うので、何週間も使わないときは、月に1度は充電します。

## F. 重ねて、体に付ける

1. **XIAO とセンサーを背中合わせ**に、厚手の両面テープで貼る（XIAO の裏とセンサーの裏が向き合う）。コードは横に折りたたむ。
2. **電池をセンサーの表側**に、両面テープで重ねる。はんだの尖った先が電池に当たらないように、間にテープを1枚はさむ（電池の袋に穴があくと発火のおそれ）。
3. 体側から **電池** → **センサー** → **XIAO** の順になるように付け、**XIAO の部品の面を外に**向ける（アンテナが体や電池でふさがれないように）。
4. ミニチャック袋に入れ、ラベルを袋の外にも貼る。
5. 面ファスナーのバンドで、**ずれないように、しっかり**体に巻く。揺れると、その揺れが雑音として記録に入ります。

## 加速度の範囲（±16 g）について

足首の着地は 10 g を超えることがあるので、`03_ble_imu` は全部の個体で加速度の範囲を **±16 g** にしています（前は ±8 g）。細かさは 1 あたり 0.000488 g（±8 g のときは 0.000244 g）になりますが、走る動きを見るには十分すぎる細かさで、困りません。範囲は通知ごとに送るので、受信画面と CSV はどちらでも正しい g を出します。

## あとでやること

- **小さくする**：センサーを白猫屋の小さい基板（13×18mm）にするなら、ピンの並びと I2C 対応をショップに確かめてから。
- **電池の持ち**：まだ測っていない。最初のランの前後で受信画面の電池の電圧を書き留め、何時間持つかを見る。

# 付録A. ファイル一覧

| ファイル | 中身 |
|---|---|
| `tools/setup.sh` | 最初に1回：Rosetta、arduino-cli、ボードのパッケージ、python のつなぎ |
| `tools/flash.sh <名前> [番号]` | コンパイルして XIAO に書き込む。`03_ble_imu 2` のように番号を付けると、名前が RFC-IMU-2 になる。`--compile-only` を付けると書き込まない |
| `tools/monitor.sh` | XIAO が出す文字を見る |
| `tools/viewer.sh` | Bluetooth の受信画面を開く |
| `tools/make_pdf.sh` | この手順書の PDF（`docs/manual.pdf`）を作り直す。付録B に今の 03_ble_imu を差し込む |
| `firmware/01_blink` | 手順2：XIAO だけで LED を点滅 |
| `firmware/02_check_lsm6dsv` | 手順5：センサーの確認（秋月電子の動作確認プログラムをもとに XIAO 用に変更、MIT License） |
| `firmware/03_ble_imu` | 手順6：1秒に100回（50回にもできる）測り、まとめて Bluetooth で送る。電池の電圧も送る。10分つながらないと眠る |
| `viewer/viewer.html` | Chrome で受けて表示、CSV で保存 |
| `viewer/test_parse.mjs` | 受信画面のパーサの確認（`node viewer/test_parse.mjs`） |

## Bluetooth で送る中身

1回の通知に、続いたサンプルを N 件まとめて入れます（N =（通知の長さ − 7）÷ 13）。Mac との MTU が 23 のままなら1件（7 + 13 = 20 バイト）、247 まで広がれば最大18件です。

| バイト | 中身 | 単位 |
|---|---|---|
| 0〜1 | 先頭のサンプルの通し番号（符号なし16ビット）。1サンプルごとに1増え、65535 の次は 0。k 件目は「先頭の番号 + k」 | 番号が飛んだら、そのぶん抜け |
| 2〜5 | 先頭のサンプルの時刻（XIAO の起動からのミリ秒、符号なし32ビット） | ms |
| 6 | 加速度の範囲（符号なし8ビット）。8 か 16 | g（±8 g / ±16 g） |
| **ここから1件13バイトを N 件**（k 件目は 7 + 13k バイト目から） | | |
| +0 | 先頭のサンプルからの時刻の差（符号なし8ビット）。そのサンプルの時刻 = 先頭の時刻 + これ | ms |
| +1〜6 | ジャイロ x・y・z（符号付き16ビット ×3） | 1 = 0.07 dps（範囲 ±2000 dps） |
| +7〜12 | 加速度 x・y・z（符号付き16ビット ×3） | ±16 g なら 1 = 0.000488 g、±8 g なら 1 = 0.000244 g |

| キャラクタリスティック | 中身 | 単位 |
|---|---|---|
| 0xFFE1（通知） | 上の表の、センサーの値 | |
| 0xFFE2（読み取り・通知） | 電池の電圧（符号なし16ビット）。10秒ごとに測り直す。USB につないでいる間は充電の電圧 | mV |

サービス 0xFFE0。すべてリトルエンディアン。センサーは 960Hz で測り、XIAO がそのうち 10ミリ秒ごと（`SAMPLE_HZ = 50` なら 20ミリ秒ごと）の最新値を読んでためておき、まとめて送ります。送るのが遅れても、ためた分（約2.5秒）までは抜けません。

## 付録B. 03_ble_imu.ino（手順6のプログラム）

```
// 手順6：センサーの値を Bluetooth（BLE）で送る
// 1秒に100回（SAMPLE_HZ を 50 にすると50回）、時刻と、ジャイロ3軸・加速度3軸を測り、
// 何回分かをまとめて1回の通知で送る。Mac の Chrome で viewer/viewer.html を開いて受ける
// 書き込みは tools/flash.sh 03_ble_imu <番号>（番号 2 なら RFC-IMU-2 という名前になる）
//
// なぜまとめるか：Mac は接続の間隔（数十ミリ秒）ごとに少ししか受け取らないので、
// 1回に1件ずつ送ると 1秒に約16回で頭打ちになる。測ったものはリングバッファにためて、取りこぼさない
//
// 送る中身（1回の通知、すべてリトルエンディアン）
//   0-1 : 先頭のサンプルの通し番号（uint16、1サンプルごとに1増える。飛んだら抜け）
//   2-5 : 先頭のサンプルの時刻（XIAO が起動してからのミリ秒、uint32）
//   6   : 加速度の範囲（uint8、g。8 か 16）。加速度の 1 が何 g かはこれで決まる（±8g は 0.000244、±16g は 0.00048
8）
//   7-  : サンプルを N 件、1件13バイト
//           +0     : 先頭のサンプルからの時刻の差（uint8、ミリ秒）
//           +1..6  : ジャイロ x, y, z（int16、1 = 0.07 dps）
//           +7..12 : 加速度 x, y, z（int16、1 = 0.000061 × 範囲/2 g）
//   N =（通知の長さ − 7）/ 13。MTU が 23 のままなら N = 1、247 なら最大 18
//   通知の中のサンプルは通し番号が1ずつ続いている（k 件目の番号 = 先頭の番号 + k）
//
// 電池の電圧は別のキャラクタリスティック 0xFFE2（読み取り＋通知、uint16、ミリボルト）で、10秒ごとに更新する
//
```

```
// センサーの設定と配線は 02_check_lsm6dsv と同じ

#include <Adafruit_TinyUSB.h>
#include <Arduino.h>
#include <Wire.h>
#include <bluefruit.h>

// 何個目か。tools/flash.sh 03_ble_imu 2 のように番号を渡すと、ここが 2 になる（渡さなければ 1）
#ifndef IMU_ID
#define IMU_ID 1
#endif
// 加速度の範囲（g）。足首の着地は 10g を超えうるので、全部の個体で 16 にしている。8 にもできる
#ifndef ACCEL_RANGE_G
#define ACCEL_RANGE_G 16
#endif
static_assert(ACCEL_RANGE_G == 8 || ACCEL_RANGE_G == 16, "ACCEL_RANGE_G は 8 か 16");
#define STR2(x) #x
#define STR(x) STR2(x)

const char* DEVICE_NAME = "RFC-IMU-" STR(IMU_ID);
constexpr uint32_t SAMPLE_HZ = 100;    // 1秒に測る回数。50 か 100
constexpr uint32_t MAX_WAIT_MS = 30;   // いちばん古いサンプルがこれだけ待ったら、満杯でなくても送る
constexpr uint32_t BATTERY_EVERY_MS = 10000; // 電池の電圧を測る間隔
constexpr uint32_t AUTO_OFF_MIN = 10;  // だれともつながらないまま、これだけたったら眠る（0 なら眠らない）

constexpr uint8_t LSM6_ADDR_GND = 0x6A;
constexpr uint8_t LSM6_ADDR_VCC = 0x6B;
uint8_t lsm6_addr = 0;

constexpr uint8_t REG_WHO_AM_I = 0x0F;
constexpr uint8_t REG_CTRL1 = 0x10;
constexpr uint8_t REG_CTRL2 = 0x11;
constexpr uint8_t REG_CTRL6 = 0x15;
constexpr uint8_t REG_CTRL8 = 0x17;
constexpr uint8_t REG_OUTX_L_G = 0x22;

constexpr uint16_t HEADER_LEN = 7;
constexpr uint16_t SAMPLE_LEN = 13;
constexpr uint16_t MAX_PAYLOAD = 244; // MTU 247 − 3

BLEService imuService(0xFFE0);
BLECharacteristic imuChar(0xFFE1);
BLECharacteristic batChar(0xFFE2);
TaskHandle_t sampleTaskHandle = NULL;

// ---- リングバッファ（測る係が入れ、送る係が出す。係は1人ずつなので鍵は要らない）----
struct Sample {
  uint16_t seq;
  uint32_t t;
  uint8_t raw[12]; // ジャイロ xyz → 加速度 xyz
};
constexpr uint32_t RING_SIZE = 256; // 100Hz で約2.5秒ぶん
Sample ring[RING_SIZE];
volatile uint32_t ringHead = 0; // 次に入れる位置（測る係だけが進める）
volatile uint32_t ringTail = 0; // 次に出す位置（送る係だけが進める）

void writeReg(uint8_t reg, uint8_t val) {
  Wire.beginTransmission(lsm6_addr);
  Wire.write(reg);
  Wire.write(val);
  Wire.endTransmission();
}

void readRegs(uint8_t startReg, uint8_t* buf, uint8_t len) {
  Wire.beginTransmission(lsm6_addr);
  Wire.write(startReg);
  Wire.endTransmission(false);
```

```
  Wire.requestFrom(lsm6_addr, len);
  for (uint8_t i = 0; i < len; i++) { buf[i] = Wire.read(); }
}

bool found(uint8_t addr) {
  Wire.beginTransmission(addr);
  return Wire.endTransmission() == 0;
}

bool lsm6_init() {
  if (found(LSM6_ADDR_GND)) lsm6_addr = LSM6_ADDR_GND;
  else if (found(LSM6_ADDR_VCC)) lsm6_addr = LSM6_ADDR_VCC;
  else return false;
  uint8_t who;
  readRegs(REG_WHO_AM_I, &who, 1);
  if (who != 0x70) return false;
  writeReg(REG_CTRL1, 0b00001001); // 加速度：高性能モード、960Hz
  writeReg(REG_CTRL2, 0b00001001); // ジャイロ：高性能モード、960Hz
  writeReg(REG_CTRL6, 0b00000100); // ジャイロの範囲：±2000 dps
  // 加速度の範囲：下の2ビットが 10 なら ±8 g、11 なら ±16 g（上の 100 はフィルターの設定で、前と同じ）
  writeReg(REG_CTRL8, ACCEL_RANGE_G == 16 ? 0b10000011 : 0b10000010);
  delay(10);
  return true;
}

// 測る係：loop() とは別の、優先度の高いタスクで、決まった間隔で測る。
// 送る係（loop）が Bluetooth の空き待ちで止まっても、測るのは止まらない。
// I2C（Wire）は setup() の後はこのタスクしか使わない
void sampleTask(void*) {
  uint16_t seq = 0;
  uint32_t n = 0;
  TickType_t base = xTaskGetTickCount();
  for (;;) {
    // n 回目を測る時刻。タイマーの刻みは 1/1024 秒なので、毎回足し算せず掛け算で出して誤差をためない
    TickType_t due = base + (TickType_t)(((uint64_t)n * configTICK_RATE_HZ) / SAMPLE_HZ);
    int32_t wait = (int32_t)(due - xTaskGetTickCount());
    if (wait > 0) vTaskDelay(wait);
    if (wait < -(int32_t)configTICK_RATE_HZ) { // 1秒以上遅れた（ふつうは起きない）：今から数え直す
      base = xTaskGetTickCount();
      n = 0;
    }

    Sample s;
    s.seq = seq++;
    s.t = millis();
    readRegs(REG_OUTX_L_G, s.raw, 12);

    uint32_t head = ringHead;
    if (head - ringTail < RING_SIZE) {
      ring[head % RING_SIZE] = s;
      __DMB(); // 中身を書き終えてから位置を進める
      ringHead = head + 1;
    }
    // 満杯なら捨てる。通し番号は進むので、受ける側で抜けが分かる
    n++;
  }
}

// ---- 電池 ----
// XIAO の電池の電圧は、1MΩ と 510kΩ で分けて P0.31 で読む。P0.14 を LOW にすると読める。
// Seeed の資料：P0.14 を HIGH にしたまま充電すると P0.31 が 3.6V の上限に届いて壊れるおそれがあるので、
// いつも LOW にしておく（ボードのパッケージは起動時に HIGH にするので、setup() の最初で LOW に直す）
void batteryInit() {
  pinMode(VBAT_ENABLE, OUTPUT);
  digitalWrite(VBAT_ENABLE, LOW);
  analogReadResolution(12); // 0〜4095
  analogSampleTime(40);     // 抵抗が大きい（約340kΩ）ので、長めに取り込む
```

```
}

uint16_t batteryMillivolts() {
  uint32_t sum = 0;
  for (int i = 0; i < 8; i++) sum += analogRead(PIN_VBAT);
  // 0〜4095 が 0〜3.6V（内部の基準 0.6V × 6）。分けた電圧を元に戻すため ×(1000+510)/510
  return (uint16_t)((sum / 8.0f) * 3600.0f / 4096.0f * 1510.0f / 510.0f);
}

void batteryUpdate() {
  uint16_t mv = batteryMillivolts();
  batChar.write16(mv);
  if (Bluefruit.connected() && batChar.notifyEnabled()) batChar.notify16(mv);
}

// 眠る：センサーを止めてから、nRF52840 を System OFF にする。
// 起きるのは、リセットボタンを1回押したとき、または USB をつないだとき（起きると最初から動く）
void sleepNow() {
  vTaskSuspend(sampleTaskHandle); // I2C を使う係を止めてから、センサーに書く
  writeReg(REG_CTRL1, 0);         // 加速度を止める
  writeReg(REG_CTRL2, 0);         // ジャイロを止める
  digitalWrite(LED_BUILTIN, HIGH); // LED を消す（この LED は HIGH で消える）
  Serial.println("sleep");
  delay(100);
  sd_power_system_off();
}

void onConnect(uint16_t conn_hdl) {
  BLEConnection* conn = Bluefruit.Connection(conn_hdl);
  if (!conn) return;
  // Mac に、大きく・こまめに受け取ってもらうよう頼む（決めるのは Mac。断られても、まとめて送るので動く）
  conn->requestPHY();
  conn->requestDataLengthUpdate();
  conn->requestMtuExchange(247);
  conn->requestConnectionParameter(12); // 15ミリ秒（単位 1.25ミリ秒。Apple の決まりで 15 より短くは頼まない）
}

void startAdvertising() {
  Bluefruit.Advertising.addFlags(BLE_GAP_ADV_FLAGS_LE_ONLY_GENERAL_DISC_MODE);
  Bluefruit.Advertising.addTxPower();
  Bluefruit.Advertising.addService(imuService);
  Bluefruit.ScanResponse.addName();
  Bluefruit.Advertising.restartOnDisconnect(true); // 切れたら、また見つけてもらえるようにする
  Bluefruit.Advertising.setInterval(32, 244);      // 単位は0.625ミリ秒
  Bluefruit.Advertising.setFastTimeout(30);
  Bluefruit.Advertising.start(0); // 0 = ずっと続ける
}

void setup() {
  batteryInit(); // P0.14 を LOW にする（上の説明）。ほかのことより先に
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(115200); // Mac につないでいなくても動く（シリアルを待たない）
  Wire.begin();
  delay(100);

  while (!lsm6_init()) {
    Serial.println("LSM6DSV16X not found. 02_check_lsm6dsv で配線を確かめてください");
    digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN)); // 速く点滅して知らせる
    delay(200);
  }

  Bluefruit.configPrphBandwidth(BANDWIDTH_MAX); // MTU 247 まで。1回の接続の間に何通も送れるように
  Bluefruit.begin();
  Bluefruit.setName(DEVICE_NAME);
  Bluefruit.Periph.setConnInterval(12, 24); // 希望：15〜30ミリ秒（単位 1.25ミリ秒）
  Bluefruit.Periph.setConnectCallback(onConnect);
```

```
  imuService.begin();
  imuChar.setProperties(CHR_PROPS_NOTIFY);
  imuChar.setPermission(SECMODE_OPEN, SECMODE_NO_ACCESS);
  imuChar.setMaxLen(MAX_PAYLOAD);
  imuChar.begin();

  batChar.setProperties(CHR_PROPS_READ | CHR_PROPS_NOTIFY);
  batChar.setPermission(SECMODE_OPEN, SECMODE_NO_ACCESS);
  batChar.setFixedLen(2);
  batChar.begin();
  batteryUpdate();

  startAdvertising();
  Serial.print("BLE advertising as ");
  Serial.print(DEVICE_NAME);
  Serial.print(", ");
  Serial.print(SAMPLE_HZ);
  Serial.print(" Hz, accel +-");
  Serial.print(ACCEL_RANGE_G);
  Serial.print(" g, battery ");
  Serial.print(batteryMillivolts());
  Serial.println(" mV");

  xTaskCreate(sampleTask, "imu", 512, NULL, TASK_PRIO_NORMAL, &sampleTaskHandle);
}

// 送る係
void loop() {
  static uint32_t lastBattery = 0;
  static uint32_t lastConnected = 0;
  uint32_t now = millis();
  if (now - lastBattery >= BATTERY_EVERY_MS) {
    lastBattery = now;
    batteryUpdate();
  }
  if (Bluefruit.connected()) lastConnected = now;
  else if (AUTO_OFF_MIN > 0 && now - lastConnected >= AUTO_OFF_MIN * 60000UL) sleepNow();

  bool subscribed = Bluefruit.connected() && imuChar.notifyEnabled();
  if (!subscribed) {
    ringTail = ringHead; // だれも受けていない間のぶんは捨てる（つないだ直後に古い値を送らない）
    delay(5);
    return;
  }

  uint32_t tail = ringTail;
  uint32_t avail = ringHead - tail;
  if (avail == 0) {
    delay(2);
    return;
  }

  BLEConnection* conn = Bluefruit.Connection(Bluefruit.connHandle());
  if (!conn) return;
  uint16_t payload = conn->getMtu() - 3;
  if (payload > MAX_PAYLOAD) payload = MAX_PAYLOAD;
  uint32_t perPacket = (payload - HEADER_LEN) / SAMPLE_LEN; // MTU 23 なら 20 バイト：1件ちょうど入る（7 + 13
= 20）

  const Sample& first = ring[tail % RING_SIZE];
  if (avail < perPacket && (uint32_t)(millis() - first.t) < MAX_WAIT_MS) {
    delay(2); // もう少しためてから送る
    return;
  }

  uint8_t pkt[MAX_PAYLOAD];
  memcpy(pkt, &first.seq, 2);
```

```
    memcpy(pkt + 2, &first.t, 4);
    pkt[6] = ACCEL_RANGE_G;
    uint16_t len = HEADER_LEN;
    uint32_t taken = 0;
    while (taken < avail && taken < perPacket) {
      const Sample& s = ring[(tail + taken) % RING_SIZE];
      uint32_t dt = s.t - first.t;
      if (dt > 255) break;                                  // 時刻の差が1バイトに入らない：次の通知へ
      if ((uint16_t)(s.seq - first.seq) != taken) break; // 抜けをまたがない：次の通知へ
      pkt[len] = (uint8_t)dt;
      memcpy(pkt + len + 1, s.raw, 12);
      len += SAMPLE_LEN;
      taken++;
    }

    // 送る口に空きがなければ notify は最大100ミリ秒待つ。その間も測る係はためつづける。
    // 送れなかったら、同じサンプルを次の回にもう一度送る
    if (imuChar.notify(pkt, len)) {
      ringTail = tail + taken;
    }
  }
```

## 付録C. 参考にしたもの

- 秋月電子通商 AE-LSM6DSV16X 商品ページ・回路図・動作確認プログラム：akizukidenshi.com/catalog/g/g130950/
- Seeed Studio XIAO nRF52840 Wiki（ピン配置、裏の BAT の位置の図、充電の電流、P0.14 の注意）：wiki.seeedstudio.com/XIAO_BLE/
- XIAO nRF52840 の回路図（電池の電圧は 1MΩ と 510kΩ で分けて P0.31 へ）：files.seeedstudio.com/wiki/XIAO-BLE/Res/260828_XIAO_nRF52840.pdf
- XIAO nRF52840 の日本の工事設計認証書（211-220207）：files.seeedstudio.com/Seeed_Certificate/documents_certificate/102010448-TELEC.pdf
- Smol Slime の作例（XIAO nRF52840 ＋ LSM6DSV）：note.com/mmtg03/n/ncabe6e5c6596